Pontificia Universidad Javeriana

Facultad de Ingeniería

Sistemas Operativos

Taller de Evaluación de Rendimiento

Martín Sanmiguel Delgado

Juan Manuel Baez Parra

Mateo Traslaviña Moreno

27 de abril. 2026

Bogotá - Colombia.

1. Introducción

El presente documento tiene como objetivo evaluar y comparar el rendimiento de distintos sistemas de cómputo operando bajo distribuciones de Linux. Para ello, se busca analizar el comportamiento del hardware al ejecutar diferentes algoritmos de multiplicación de matrices y, posteriormente, establecer una correlación directa entre los tiempos de ejecución obtenidos y las especificaciones técnicas de cada máquina.

2. Descripción de los sistemas de computo

Para la realizar el experimento se utiliza 6 sistemas de cómputo Linux, con cuatro arquitecturas de hardware diferentes. La Tabla 1 a continuación, se muestran los componentes de hardware de cada uno de los equipos utilizados:

ID	CPU	Núcleos lógicos	Frec. Máx. (GHz)	L1d/L1i (KiB)	L2 (MiB)	L3 (MiB)	RAM (GiB)	Tipo
PC-1	Intel Core i7-6500U	4	3.1	64/64	0.5	4	7.8	**Portátil**
PC-2	Intel Core i7-6500U	4	3.1	64/64	0.5	4	7.8	**Portátil**
Escritorio-1	Intel Core i7-6700T	8	3.6	128/128	1.0	8	7.8	**Desktop**
Escritorio-2	Intel Core i7-6700T	8	3.6	128/128	1.0	8	7.8	**Desktop**
PC-Martin	Intel Core i7-13650HX	20	4.9	544/704	11.5	24	14.8	**Portátil**
VM	Intel Xeon Gold 6240R (VMware)	4 (vCPU)	2.4	128/128	4.0	35.8	12	**VM**

Tabla 1. Configuración de los diferentes Sistemas de Cómputo usados en el experimento.

Además, todas las máquinas que utilizamos tenían sistema operativo Linux Ubuntu

3. Diseño experimental

Para generar una comparación justa y mitigar el ruido computacional (*OS Jitter*) [1], se determinó ejecutar cada configuración de la evaluación múltiples veces. Este fenómeno ocurre porque, en los sistemas operativos basados en Linux, el *Kernel* gestiona interrupciones de *hardware* y procesos en segundo plano que pueden generar alteraciones en los tiempos de procesamiento. Adicionalmente, para fortalecer el rigor estadístico del estudio, se decidió aplicar el principio de la Ley de los Grandes Números [2]. Por consiguiente, cada algoritmo se ejecutó 30 veces para cada tamaño de matriz y cantidad de CPUs solicitadas. Finalmente, al promediar esta muestra, se logra aislar el tiempo real de cálculo matemático, garantizando así la validez y precisión de los datos analizados.

Para realizar la evaluación de rendimiento se utilizaron dos algoritmos:

- **Algoritmo 1:**

  Consiste en el método tradicional de multiplicar matrices, el cual opera multiplicando cada elemento de la fila de la primera matriz por cada elemento de la columna de la segunda matriz. Durante la evaluación de rendimiento, este método fue denominado como "Filas por columnas (FxC)".

  Al analizar este algoritmo desde la óptica del rendimiento, se concluye que es computacionalmente ineficiente debido a la forma en que el lenguaje C estructura los datos en la memoria, utilizando el principio conocido como Row-Major Order o almacenamiento por filas [3] [4]. Al recorrer la segunda matriz por columnas, el sistema operativo consume más tiempo adquiriendo la información, ya que tiene que hacer recorridos adicionales a través de la jerarquía de memoria al tener que traer datos de la memoria principal, violando el principio de localidad espacial y por ende, se consume más tiempo realizando la operación. En la figura 1 se puede observar cómo es que realmente los datos de los arreglos multidimensionales son almacenados. Así que se pueden esperar mayores tiempos de procesamiento para este algoritmo.

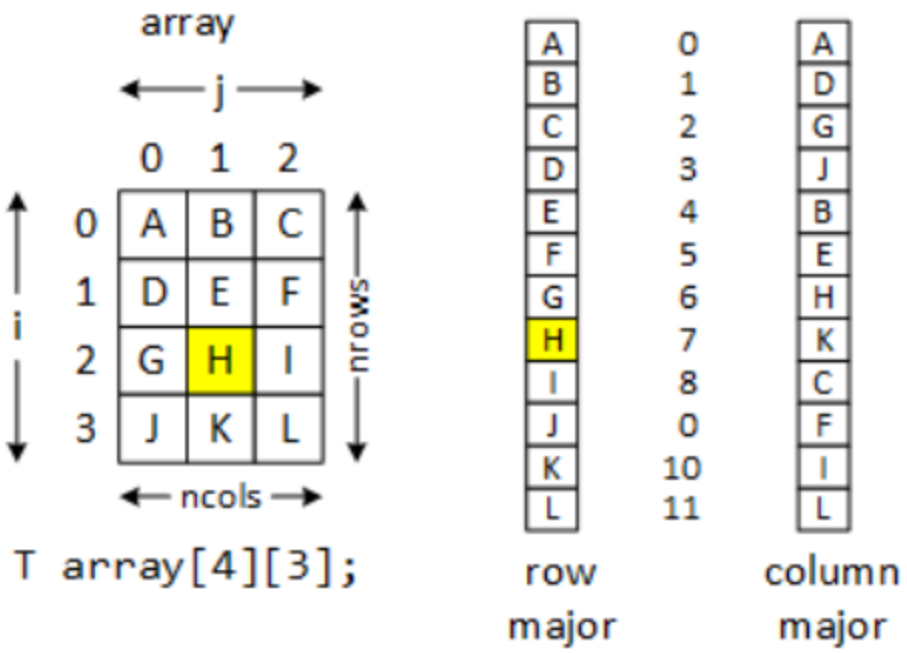

Figura 1: imagen tomada de [5] que representa Row-Major Order o almacenamiento por filas demostrando como el lenguaje C, almacena los datos de los arreglos multidimensionales

- **Algoritmo 2:** El algoritmo es una variante en donde representa una optimización del primer algoritmo en donde se tiene en cuenta la jerarquía de memoria del computador. Durante la evaluación, este método será referido como "Filas por transpuestas (FxT)".

  Este algoritmo maximiza el uso del principio de localidad espacial ya que, al transponer primero la segunda matriz, sus columnas se convierten en filas se garantiza que la información requerida por el procesador se encuentre en posiciones de memoria contiguas. De esta manera, se minimizan las consultas del sistema operativo a la memoria principal, así que se esperar un tiempo significativamente menor en comparación con el algoritmo FxC.

Antes de detallar las variables experimentales, es importante establecer la distinción entre los dos mecanismos de concurrencia utilizados en la evaluación de rendimiento:

- **Hilos (POSIX):** Un hilo es una secuencia de instrucciones que puede ejecutarse de forma independiente dentro de un programa [4]. Se caracterizan por operar dentro del mismo espacio de memoria del proceso padre, lo que les permite compartir información. Esta arquitectura minimiza el overhead [5], que se define como las tareas administrativas realizadas por el sistema operativo durante la concurrencia.

- **Procesos (Fork):** Representan instancias de ejecución independientes. A diferencia de los hilos, la llamada al sistema fork duplica el estado del proceso padre, asignando a cada proceso hijo un espacio de memoria totalmente aislado, lo que incrementa considerablemente la carga administrativa del sistema operativo y el consumo de memoria principal.

Cabe destacar que, aunque existen estándares de paralelismo de alto nivel ampliamente utilizados en la industria como OpenMP (que abstraen la creación de hilos a nivel de compilador), para este taller se seleccionó el uso explícito de POSIX Threads y Fork. Esta decisión se tomó con el objetivo de mantener un control de bajo nivel sobre el hardware y poder observar directamente el impacto de la gestión de memoria y el cambio de contexto por parte del sistema operativo.

Adicionalmente, para evaluar el comportamiento de estos algoritmos bajo distintas cargas de trabajo y niveles de concurrencia, el diseño del experimento contempla las siguientes variables:

Variable	Valores
Tamaño de las matrices	512x512 1024x1024 2048x2048
Nivel de concurrencia	1,4,8,16 CPU(s)
Mecanismos de paralelismo	POSIX Frok

Tabla 2. Variables utilizadas en la evaluación de rendimiento

4. Recolección y Procesamiento de la Información

Dado que los sistemas operativos de propósito general ejecutan múltiples procesos en segundo plano que pueden alterar el rendimiento de la CPU en un momento dado, por lo tanto, se realizó el experimento varias veces para mitigar este tipo de variaciones:

Para garantizar la integridad y repetibilidad del experimento, la recolección se dividió en dos fases automatizadas:

Fase 1: Ejecución (Script Perl): Se utilizó el script lanzador.pl para automatizar la ejecución de todas las pruebas en las cuatro máquinas. El script se encargó de compilar los binarios y ejecutar cada combinación exacta de variables (algoritmo, tamaño de matriz e hilos) 30 veces consecutivas. Los tiempos individuales de cada iteración se exportaron automáticamente a archivos de texto plano (.dat), generando un gran volumen de datos crudos.

Fase 2: Consolidación y visualización (Script Python): Para evitar errores de manipulación manual humana, se desarrolló un script de procesamiento de datos en Python utilizando las librerías Pandas, Seaborn y Matplotlib. Este código recorrió el árbol de directorios de las 4 máquinas, leyó cada archivo .dat y calculó el promedio aritmético exacto de las 30 iteraciones. Finalmente, el programa exportó los datos consolidados a archivos CSV y genero automáticamente las gráficas comparativas que sustentan el análisis posterior.

5. Análisis de Resultados

En esta sección se evalúa el impacto de las distintas variables de hardware y de software sobre los tiempos de ejecución. El análisis se fundamenta en el contenido obtenido por los Scripts descritos en la sección anterior

5.1 Ejecución Secuencial

El análisis de rendimiento inicia con la evaluación de los algoritmos bajo un entorno estrictamente secuencial, solicitando una única unidad de ejecución (CPU).

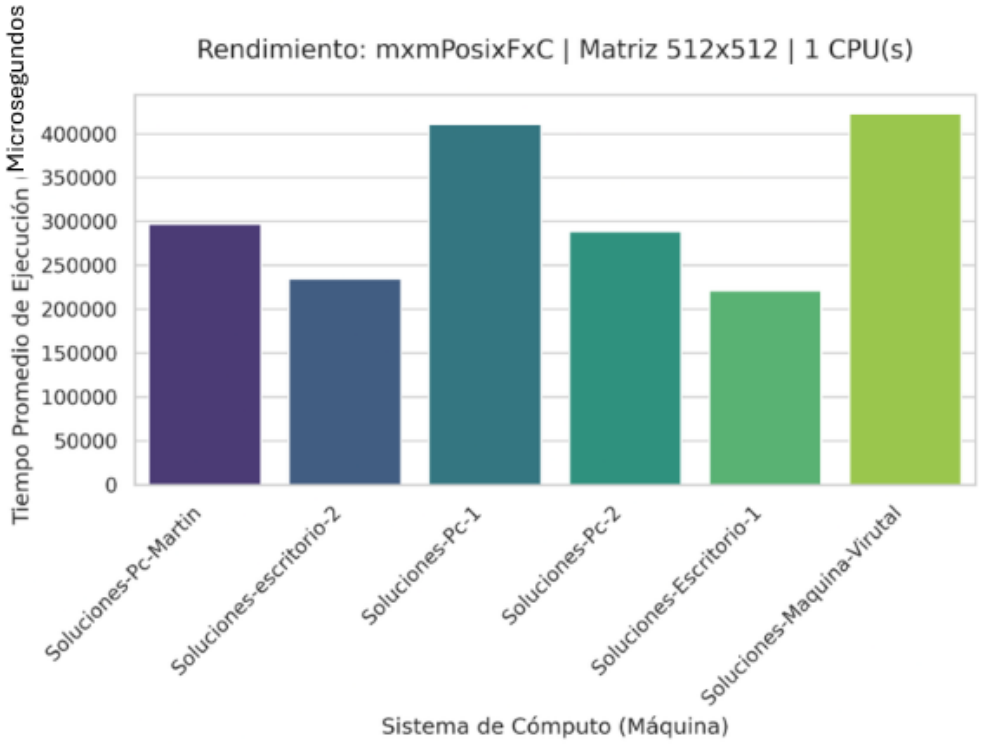

Figura 2: Tiempo de ejecución en microsegundos de la multiplicación de una matriz de 512x512 utilizando el mecanismo de Fork, comparando los distintos sistemas de cómputo.

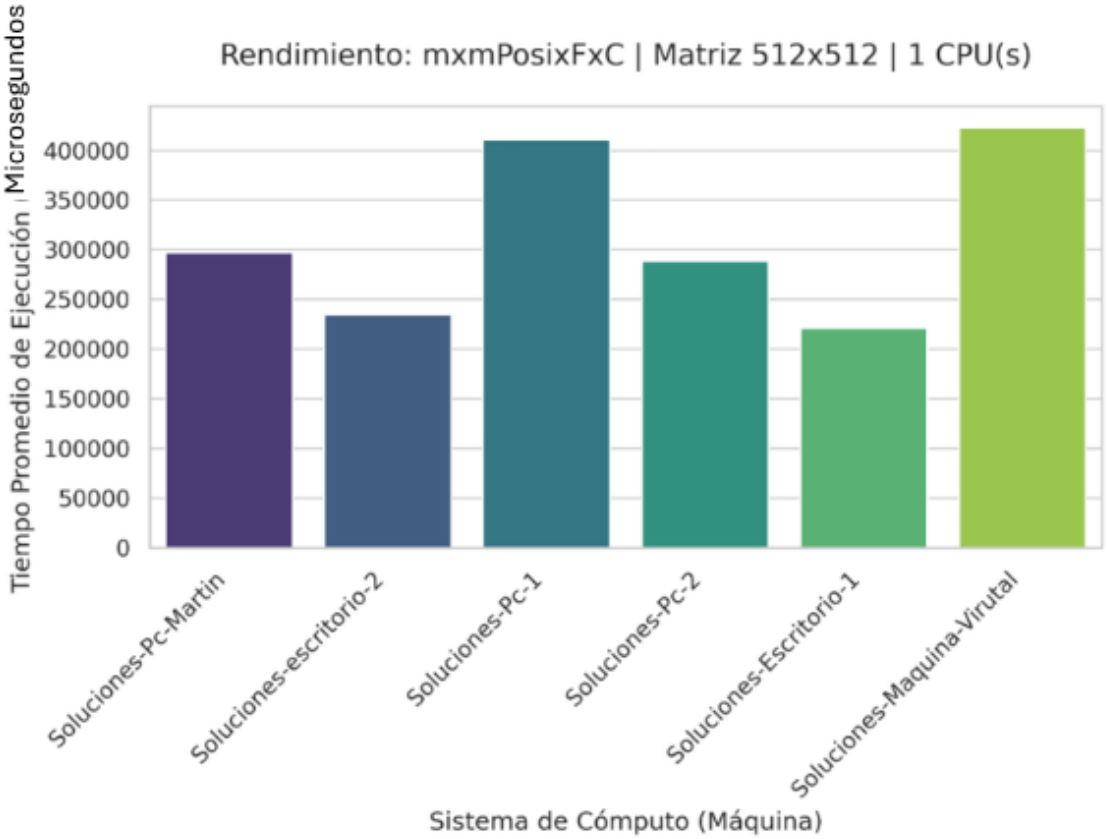

Figura 3: Tiempo de ejecución en microsegundos de la multiplicación de una matriz de 512x512 utilizando el mecanismo de POSIX, comparando los distintos sistemas de cómputo.

maquina	Tiempo Fork (μ s s)	Tiempo POSIX (μ s s)
Soluciones-Pc-Martin	325845.03	296904.33
Soluciones-escritorio-2	219751.56	235134.8
Soluciones-Pc-1	322035.06	411505.96
Soluciones-Pc-2	292692.36	288378.83
Soluciones-Escritorio-1	222042.36	221354.16
Soluciones-Maquina-Virutal	413081.73	423467.76

Promedio	299241	242212.5

Tabla 3. Datos de tiempo de ejecución de la multiplicación de una matriz 512x512 por sistema de cómputo utilizando Fork y POSIX

Al observar el comportamiento de la carga de trabajo inicial 512x512 en las Figuras 2 y 3, se realizan tres Observaciones:

- **Superioridad del Hardware de Escritorio:** Los sistemas de cómputo con mayor rendimiento fueron los equipos de escritorio (Escritorio-1 y Escritorio-2). Esto se debe a que las arquitecturas de escritorio no sufren de las limitaciones térmicas y de energía que restringen a los equipos portátiles (como PC-Martin, PC-1 y PC-2). Al tener mejor disipación de calor y estar conectados a la corriente, los procesadores de escritorio pueden mantener su rendimiento al máximo de manera sostenida.

- **El Costo de rendmiento de la Virtualización:** Se evidencia de manera contundente que la Máquina Virtual fue el entorno con peor rendimiento, siendo un 53% más lenta al usar Fork y un 52% más lenta usando POSIX en comparación con los equipos de escritorio (Escritorio-1 y Escritorio-2). Este resultado fue generado por el overhead introducido por el Hipervisor. En un entorno virtualizado, cada instrucción del procesador y cada acceso a la memoria RAM debe atravesar una capa extra de abstracción de software antes de llegar al hardware real, lo que introduce una latencia significativa en tareas como cálculos matemáticos.

- **Eficiencia de Hilos vs Procesos:** Finalmente, al promediar los tiempos globales, se nota que la implementación con POSIX fue aproximadamente un 20% más veloz que la implementación con Fork. Esto ocurre porque la llamada al sistema fork() es mucho más "pesada" el sistema operativo que la llamada de POSIX, ya que la creacion de procesos obliga a duplicar las tablas del Bloque de Control de Proceso (PCB) completo. Por el contrario, pthread_create() (POSIX) genera un hilo ligero que comparte el espacio de memoria programa principal, ahorrando tiempo.

El análisis procede a evaluar el comportamiento de los sistemas bajo un incremento en la carga de trabajo, pasando a la ejecución de la matriz de 1024x1024.

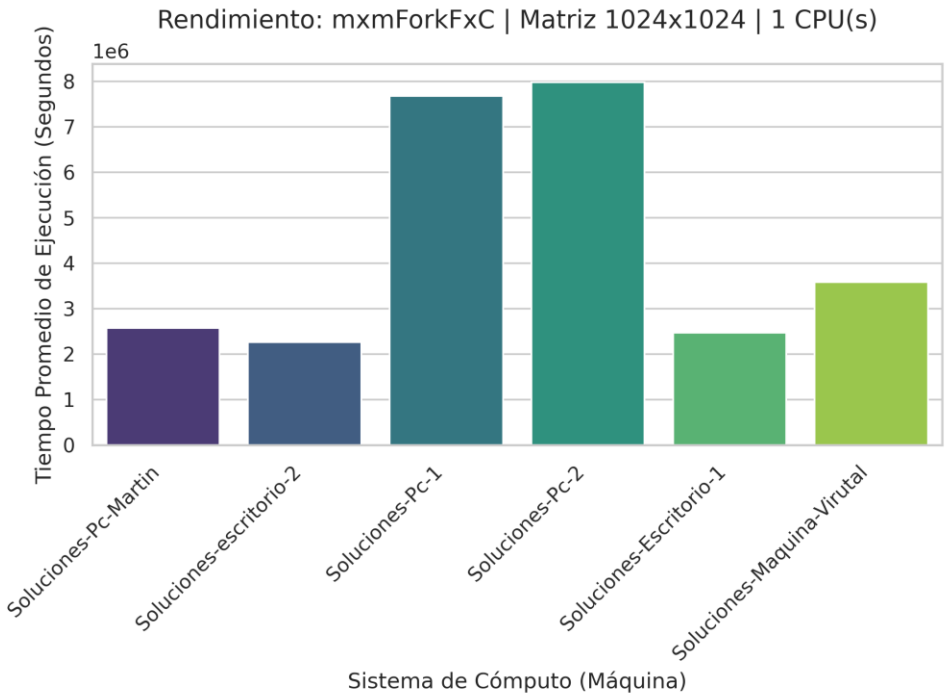

Figura 4: Tiempo de ejecución en segundos de la multiplicación de una matriz de 1024x1024 utilizando el mecanismo de fork, comparando los distintos sistemas de cómputo.

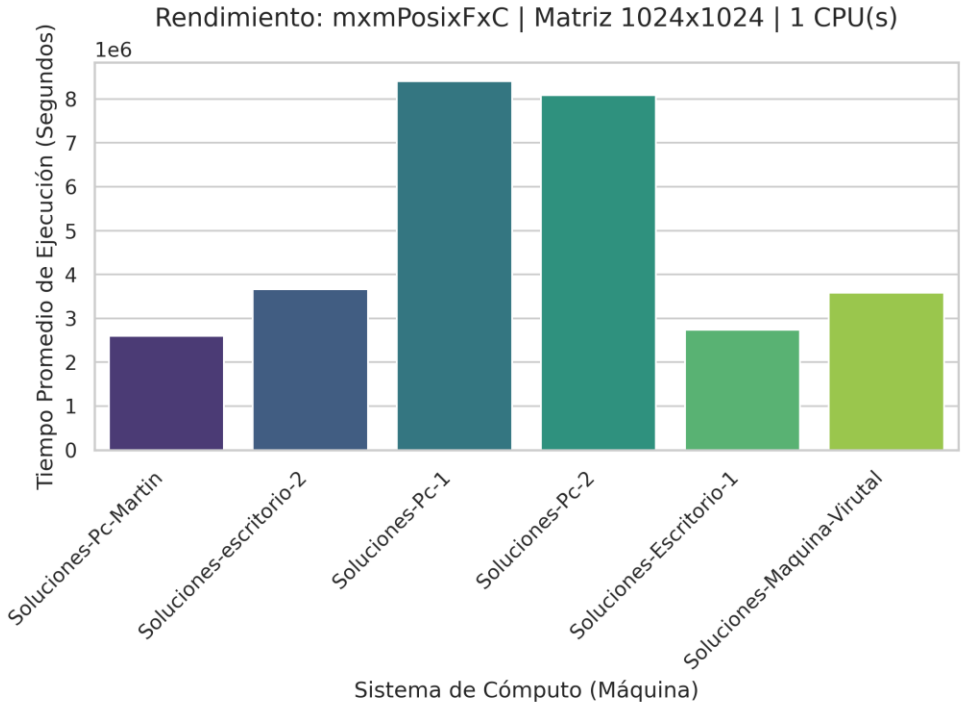

Figura 5: Tiempo de ejecución en segundos de la multiplicación de una matriz de 1024x1024 utilizando el mecanismo de POSIX, comparando los distintos sistemas de cómputo.

maquina	Tiempo Fork (μ s)	Tiempo POSIX (μ s)
Soluciones-Pc-Martin	2572595.46	2604801.0
Soluciones-escritorio-2	2257533.26	3662123.96
Soluciones-Pc-1	7671777.5	8409214.5
Soluciones-Pc-2	7980329.26	8090583.03
Soluciones-Escritorio-1	2469934	2736281.2
Soluciones-Maquina-Virutal	3576913.36	3588558.1
Promedio	4421513.5	4848593.33

Tabla 4. Datos de tiempo de ejecución de la multiplicación de una matriz 1024x1024 por sistema de cómputo utilizando Fork y POSIX

Al escalar la carga de trabajo a la dimensión de 1024x1024, los datos empíricos confirman que el comportamiento arquitectónico observado en la línea base 512x512 se mantiene constante. Los sistemas de escritorio conservan su liderazgo frente a las limitaciones térmicas de los portátiles, la abstracción de la máquina virtual sigue representando un cuello de botella, y aunque se suponia que teoría general dice que POSIX (hilos) debería ser más rápido porque crear un hilo es más ligero para la memoria principal que crear un proceso. Sin embargo, si tus datos muestran que Fork fue más rápido, significa que fork tuvo una mayor eficiencia a la hora de la ejecucion generando que Fork fuera aproximadamente un 10% más eficiente que POSIX

El análisis procede a evaluar el comportamiento de los sistemas bajo un incremento en la carga de trabajo, pasando a la ejecución de la matriz de 2048x2028.

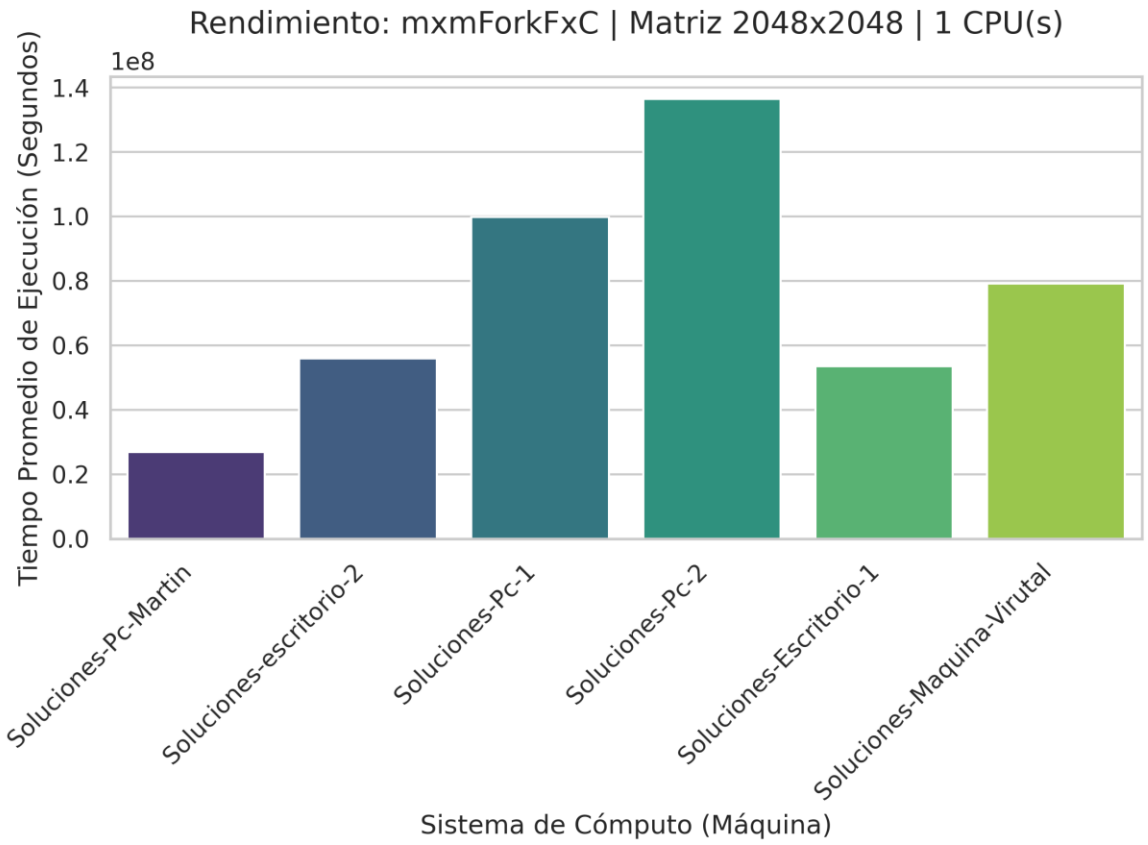

Figura 6: Tiempo de ejecución en segundos de la multiplicación de una matriz de 2048x2048 utilizando el mecanismo de fork, comparando los distintos sistemas de cómputo.

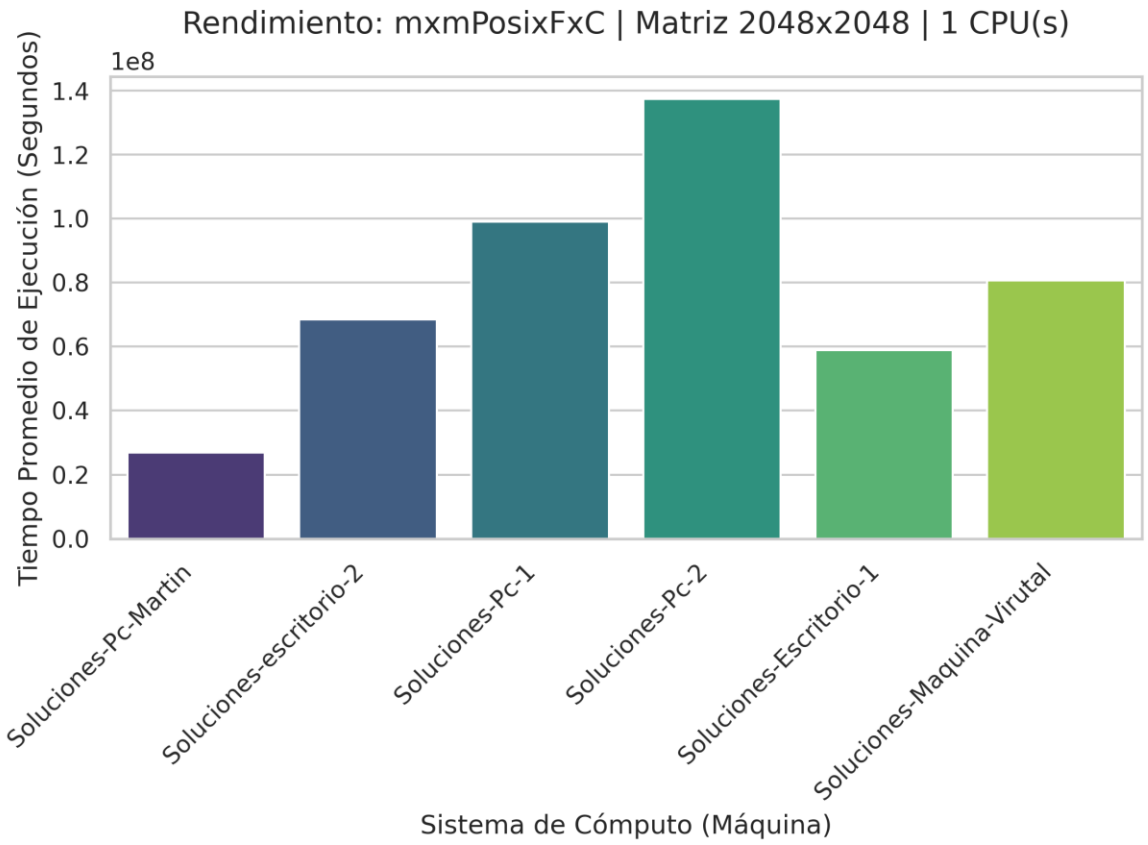

Figura 7: Tiempo de ejecución en segundos de la multiplicación de una matriz de 2048x2048 utilizando el mecanismo de POSIX, comparando los distintos sistemas de cómputo.

maquina	Tiempo Fork (μ s)	Tiempo POSIX (μ s)
Soluciones-Pc-Martin	26872231.7	655542.06
Soluciones-escritorio-2	55972042.23	1766055.96
Soluciones-Pc-1	99876021.63	5835033.3
Soluciones-Pc-2	136585518.9	634849.43
Soluciones-Escritorio-1	53572751.0	863305.43

Soluciones-Maquina-Virutal	79277194.03	679935.83
Promedio	75,359,293	1,739,120

Tabla 4. Datos de tiempo de ejecución de la multiplicación de una matriz 1024x1024 por sistema de cómputo utilizando Fork y POSIX

FORK

En la implementación basada en **FORK**, los resultados obtenidos para la multiplicación de matrices de tamaño 2048×2048 evidencian tiempos de ejecución elevados y altamente dispersos entre los distintos sistemas de cómputo, con valores que oscilan aproximadamente entre 26.8 millones y 136.5 millones de microsegundos, y un promedio cercano a 75.3 millones de microsegundos.

Esta variabilidad no es aleatoria, sino que está directamente relacionada con la forma en que los procesos son gestionados por el sistema operativo. A diferencia de los hilos, cada llamada a fork() genera un proceso independiente con su propio espacio de memoria, lo que implica una duplicación lógica del contenido mediante mecanismos como copy-on-write.

Sin embargo, dado que la multiplicación de matrices implica un acceso intensivo y constante a grandes volúmenes de datos, este mecanismo pierde efectividad y se producen copias reales en memoria, incrementando significativamente el tiempo de ejecución. Este efecto se amplifica debido al tamaño de la matriz (2048×2048), donde la complejidad computacional y el volumen de datos obligan al sistema a realizar múltiples accesos a memoria principal, generando un cuello de botella.

A nivel de hardware, los equipos con mayores recursos, como el PC-Martin con más núcleos y mayor caché, logran mitigar parcialmente este impacto, mientras que sistemas más limitados como PC-2 o entornos virtualizados presentan los peores tiempos debido a saturación de núcleos, menor capacidad de caché y mayor overhead del sistema operativo. En conjunto, los datos reflejan que FORK no escala adecuadamente para cargas de trabajo intensivas en memoria, ya que el costo de gestión de procesos y duplicación de datos supera ampliamente los beneficios del paralelismo.

POSIX

Por otro lado, la implementación mediante **POSIX Threads** presenta un comportamiento notablemente más eficiente y estable, con tiempos de ejecución significativamente menores, que

varían entre aproximadamente 634 mil y 5.8 millones de microsegundos, y un promedio cercano a 1.74 millones de microsegundos.

Esta mejora sustancial se explica por la naturaleza de los hilos, los cuales comparten el mismo espacio de memoria dentro del proceso, eliminando la necesidad de duplicar datos y permitiendo un acceso directo y concurrente a las matrices. En el contexto de matrices grandes como 2048×2048, esta característica resulta fundamental, ya que maximiza el aprovechamiento de la jerarquía de memoria y la localidad espacial, especialmente cuando se utiliza un algoritmo optimizado como el de filas por transpuesta.

Además, el menor costo de creación y gestión de hilos reduce significativamente el overhead del sistema operativo, permitiendo que la mayor parte del tiempo de ejecución se destine al cálculo efectivo.

A nivel de hardware, aunque existen diferencias entre los sistemas, estas son menos pronunciadas que en FORK, lo que indica una mayor estabilidad del modelo de ejecución; incluso la máquina virtual muestra un rendimiento competitivo debido a la menor dependencia de operaciones costosas a nivel de kernel.

En síntesis, los resultados demuestran que POSIX Threads no solo reduce drásticamente los tiempos de ejecución, sino que también escala de manera más eficiente con el tamaño del problema y se adapta mejor a las limitaciones del hardware, consolidándose como la alternativa más adecuada para este tipo de aplicaciones de cómputo intensivo.

5.2 Ejecucion en paralelo

El siguiente análisis de rendimiento se centra en la ejecución concurrente, evaluando el comportamiento de los sistemas al distribuir la carga de la matriz de 2048x2048 en 4, 8 y 16 unidades de ejecución (CPUs). Las Figuras 8 y 9 ilustran cómo cada arquitectura responde a la inyección de paralelismo bajo los mecanismos de POSIX y Fork

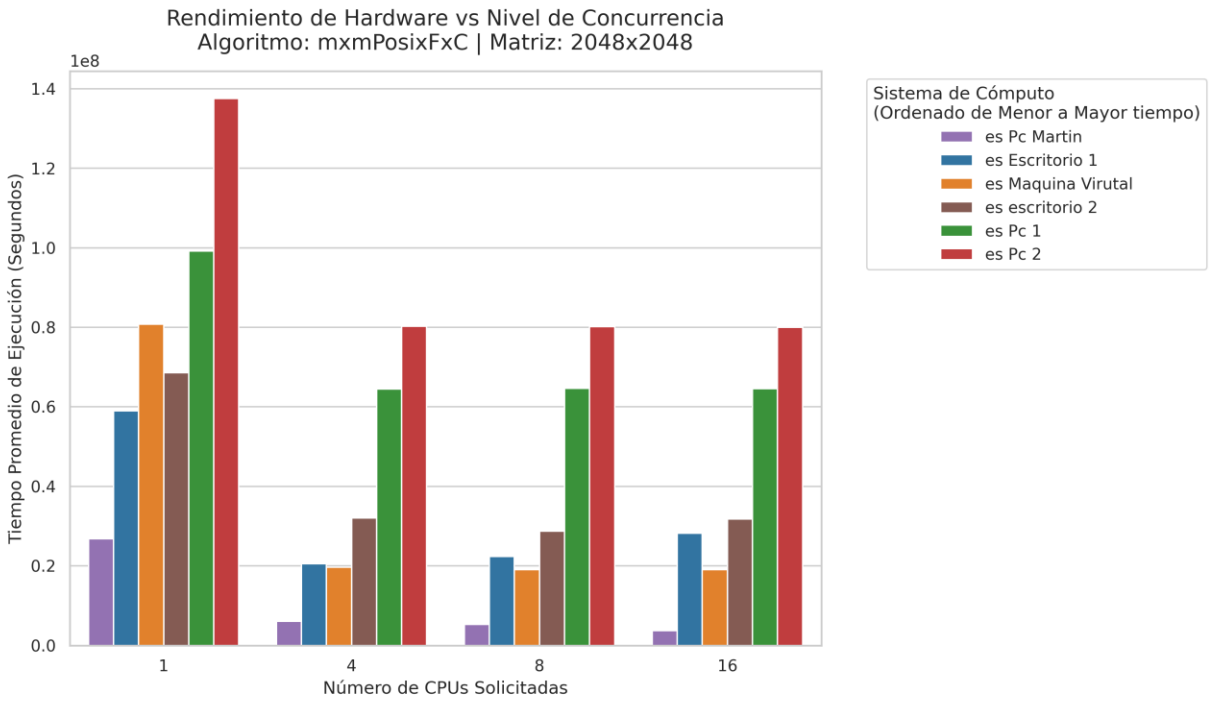

Figura 8: grafica donde el eje y aparece el tiempo de ejecución de la multiplicación de matrices 2048x2048 utilizando el algoritmo POSIX y el eje x se pueden ver cada una de las máquinas y cada una de la cantidad de hilos

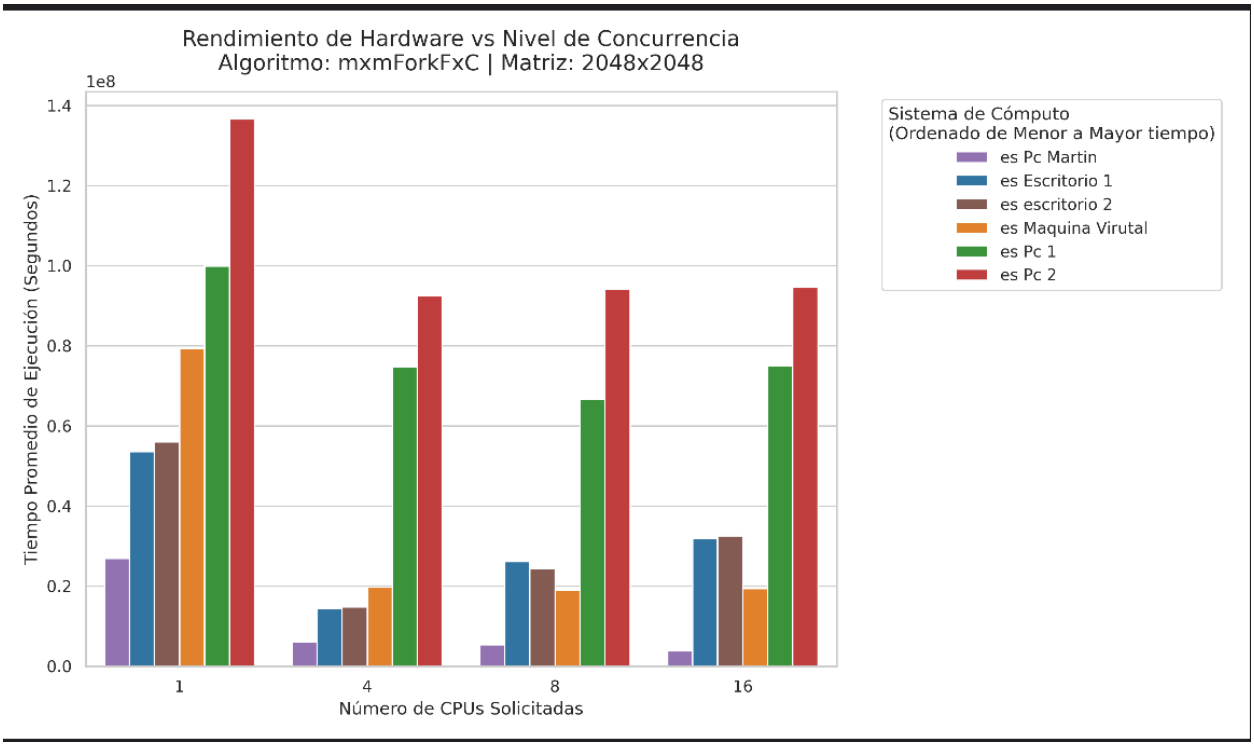

Figura 9: grafica donde el eje y aparece el tiempo de ejecución de la multiplicación de matrices 2048x2048 utilizando el algoritmo Fork y el eje x se pueden ver cada una de las máquinas y cada una de la cantidad de hilos:

- **Saturación Equipos Portátiles Estándar:** Con la excepción del equipo de PC-martin, las máquinas portátiles tradicionales presentan los tiempos más deficientes de toda la comparativa. Al contar con solo 4 núcleos físicos y ser sometidas a 16 unidades de ejecución, estos equipos entran en un estado de alta concurrencia simulada, aplanando el crecimiento de su rendimiento.

- **PC-martin supera el rendimiento de todos:** El equipo portátil de alto rendimiento mantiene su posición como el sistema más rápido en todos los escenarios. Al poseer 20 núcleos lógicos, es la única arquitectura del experimento capaz de procesar 16 tareas de manera estrictamente paralela. El sistema operativo realiza las tareas sin porblema aprovechando los nucleos de ejecucion y la memoria caché, le permite resolver la matriz en una fracción del tiempo de sus competidores.

- **El rendimiento máquina virtual 16 CPUs:** El fenómeno más destacable en el nivel de 16 unidades de ejecución es que la Máquina Virtual logró superar el rendimiento de los equipos de escritorio físicos. Aunque los equipos de escritorio cuentan con 8 núcleos y la VM solo expone 4 v CPUs, la explicación radica en el hardware de la máquina que la aloja. La VM está alojada sobre un procesador de servidor (Intel Xeon Gold), el cual posee un ancho de banda de memoria y una Caché L3 de muy grande. Cuando los equipos de escritorio intentan gestionar 16 procesos para una matriz este se satura. Por el contrario, el procesador Xeon absorbe eficientemente las múltiples peticiones de memoria concurrentes gracias a su arquitectura de servidor, mitigando el impacto del embotellamiento y superando a los equipos de escritorio físicos.

5.3. Impacto de la cantidad de hilos, cantidad de trabajo y escalabilidad

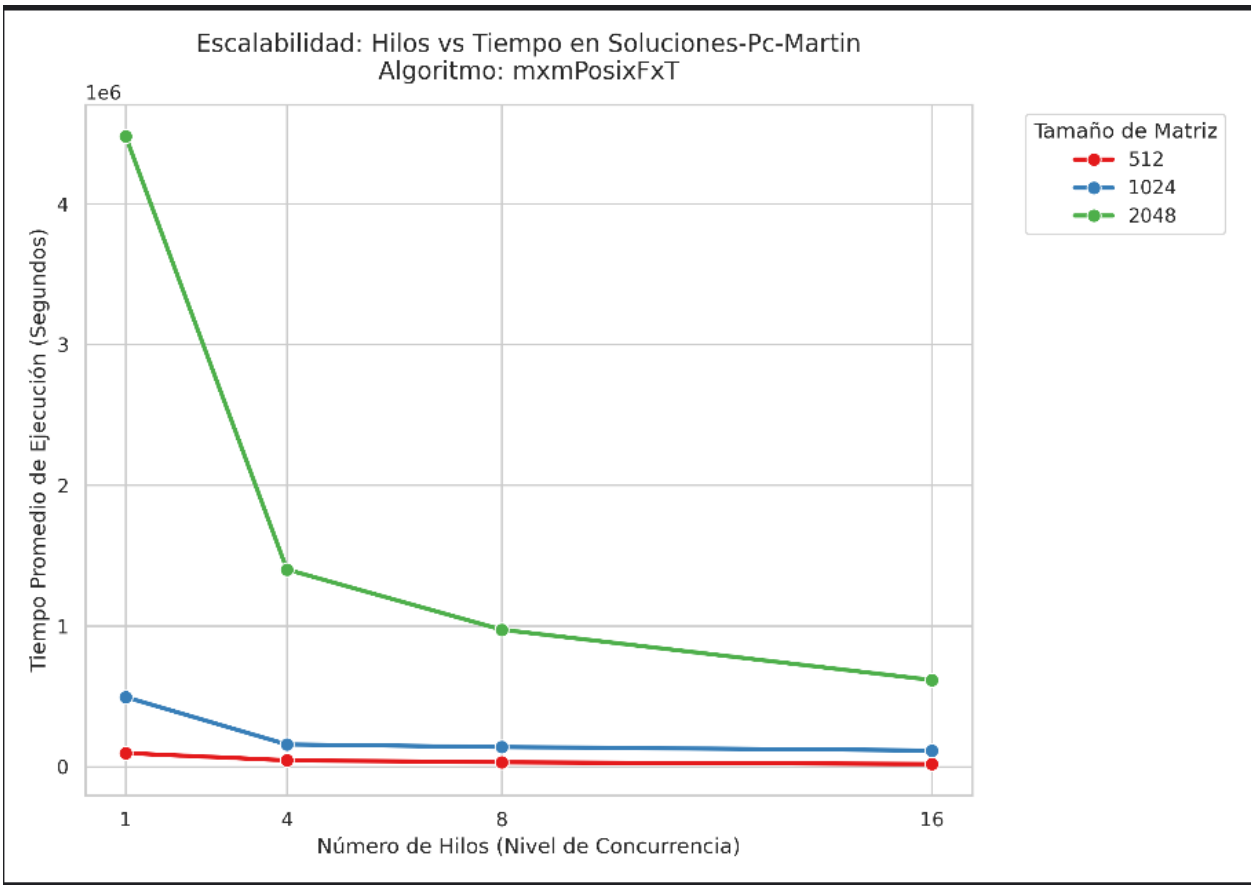

Figura 10: en esta grafica podemos observar el Overhead con claridad en el computador PC-martin, esta grafica compara los tiempos con diferentes tamaños de matrices y diferente cantidad de hilos

En la figura 10 podemos observar el cambio de contexto o Overhead [5], este existe ya que para que la concurrencia sea posible, el núcleo del sistema operativo debe intervenir para crear los hilos, asignar recursos y realizar constantes cambios de contexto (osea consumir tiempo en tareas administrativas) y cuanto la tarea es muy pequeña como cuando se realiza la multiplicación de matrices de tamaño 512x512 (Figura 2) pues el sistema operativo consume demasiado tiempo creando y gestionando los 16 hilos (línea naranja), es por esto que se puede observar que es una línea casi recta, osea que aunque se tiene una eficacia muy grande con la gran cantidad de hilos la mayoría del tiempo de ejecucion es consumido por el Overhead. En este caso, el sistema operativo gasta más tiempo organizando el trabajo que los hilos ejecutándolo.

Por el contrario, en la matriz de 2048x2048, el panorama cambia drásticamente. Dado que la carga de trabajo es masiva, el costo del overhead se vuelve despreciable frente al volumen de cálculos realizados. Esto genera una escalabilidad positiva, observable en la caída pronunciada de la curva: al pasar de 1 a 4 hilos el tiempo de ejecución se reduce de forma exponencial, manteniendo una mejora significativa hasta los 16 hilos. Aquí, el sistema logra una utilización óptima del hardware, ya que el esfuerzo administrativo de la concurrencia se compensa con una reducción sustancial en el tiempo de procesamiento total.

5.2 Análisis de Saturación de Hardware (caso PC-2, Escritorio-2, VM)

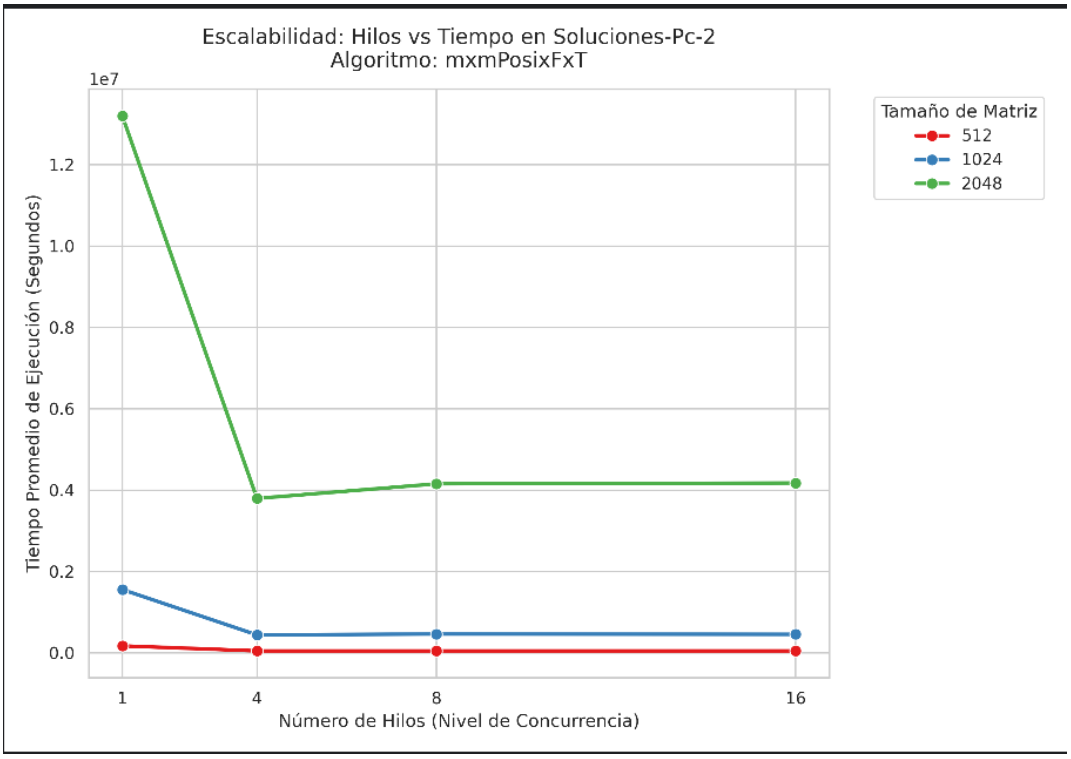

Figura 11: en esta figura evidenciar lo que pasa cuando al sistema de cómputo se le piden usar más hilos de los que tiene, esta es la gráfica de escalabilidad del Pc 2

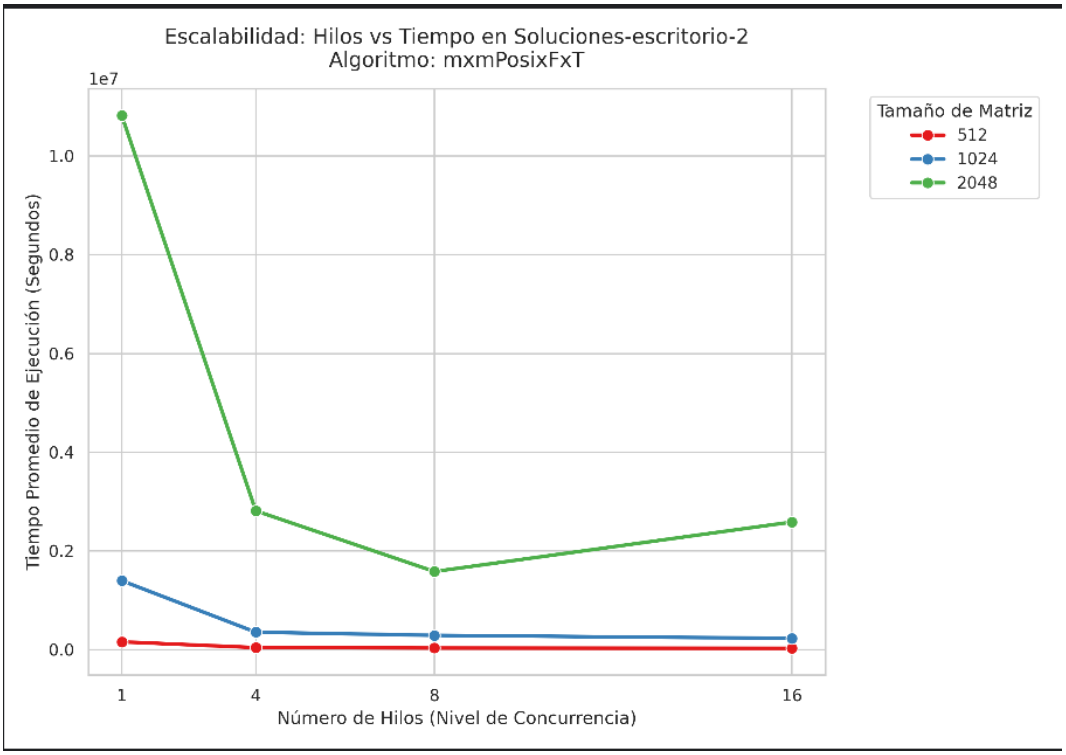

Figura 12: En esta figura evidenciar lo que pasa cuando al sistema de cómputo se le piden usar más hilos de los que tiene, esta es la gráfica de escalabilidad del escritorio 2

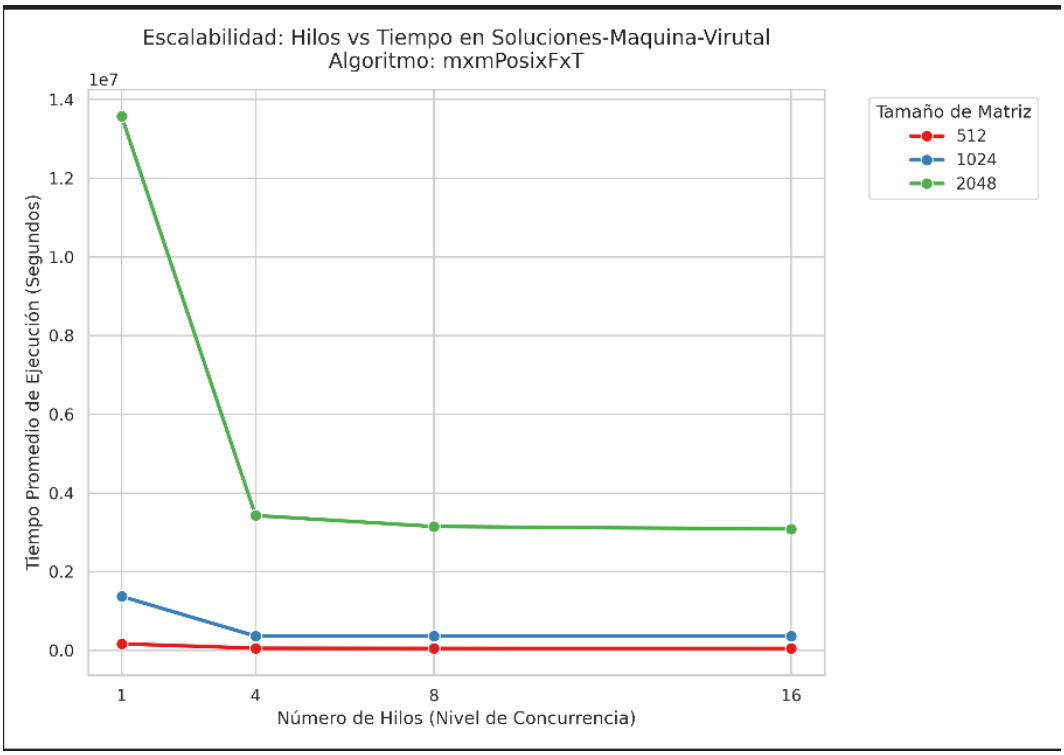

Figura 13: en esta figura evidenciar lo que pasa cuando al sistema de cómputo se le piden usar más hilos de los que tiene, esta es la gráfica de escalabilidad de la máquina virtual

Unos casos muy interesantes son los que se presentan cuando se observan las gráficas de escalabilidad del PC-2, ESCRITORIO-2 Y MAQUINA-VIRTUAL (figura 11, 12 y 13). Primero podemos analizar el caso del PC-2, ya que el caso de este sistema de cómputo es similar al resto. Este equipo a diferencia otros equipos con mayor capacidad (como el Pc-Martin), este sistema cuenta con una limitación física de 4 hilos de ejecución.

En la Figura 12, se puede observar que el tiempo de ejecución disminuye correctamente al pasar de 1 a 4 hilos. Sin embargo, al incrementar la carga a 8 y 16 hilos, la curva se vuelve completamente plana. Este comportamiento se analiza bajo los siguientes puntos:

Límite de Paralelismo: Una vez que el software solicita más hilos (8 o 16) de los que el hardware puede procesar simultáneamente (4), el procesador alcanza su punto máximo de rendimiento, ya que no hay unidades de ejecución libres para procesar las nuevas tareas en paralelo, así que se llega a un límite físico del hardware.

Gestión por Tiempo Compartido: Para manejar los 16 hilos en solo 4 núcleos lógicos, el Sistema Operativo debe recurrir a la alternando trozos de tiempo del procesador entre los 16 hilos. Aunque el programa intenta ejecutar 16 tareas a la vez, el hardware solo está procesando 4 en cualquier instante dado (o se muestra con un rendimiento similar a 4).

Estabilización del Rendimiento: El hecho de que la línea sea plana indica que el sistema es y que esta no "baje" confirma la Ley de Amdahl [7][8]: el límite de la aceleración está dictado por la parte del hardware que no puede ser paralelizada más allá de sus recursos físicos o en palabras más simples el hardware que no puede ofrecer más rendimiento esta generando un cuello de botella.

Estas tres cosas son las que ocurren en el resto de los sistemas generando que la mejora de eficiencia se detenga cuando se llega al límite físico del sistema de computo.

5.3 Comparación POSIX y FORK

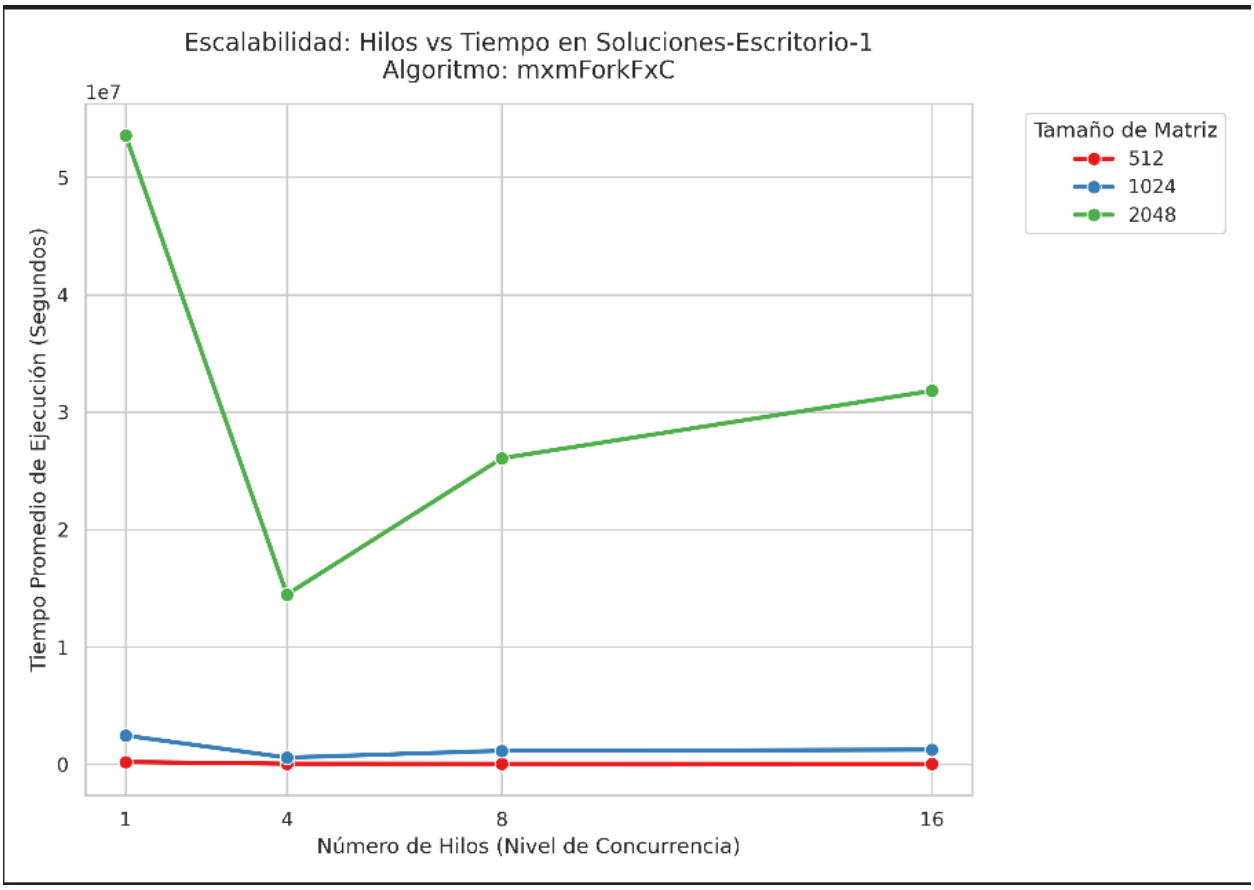

Figura 14: grafica de escalabilidad de la maquina escritorio 1 FxC con fork

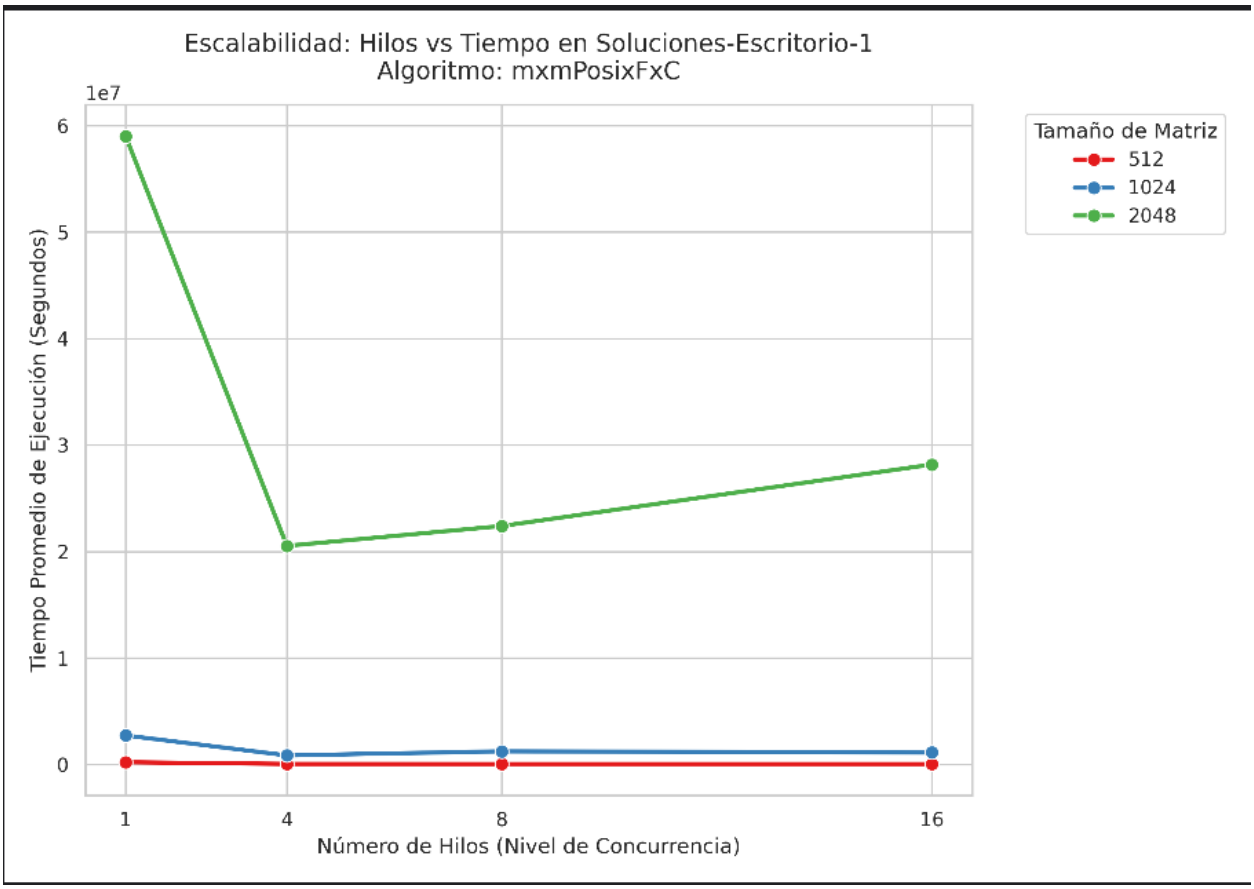

Figura 15: grafica de escalabilidad de la maquina escritorio 1 FxC con POSIX

Al contrastar los resultados obtenidos entre las implementaciones de POSIX Threads y la llamada al sistema Fork (visualizados en las Figuras 14 y 15), se evidencia una diferencia sustancial en el rendimiento, especialmente bajo cargas de trabajo elevadas (2048x2048).

Como se puede ver POSIX es más rápido que FORK y esto pasa por La superioridad técnica de POSIX se genera ya que los hilos operan dentro del mismo espacio de memoria del proceso padre. Esto permite que la comunicación y el acceso a las matrices sean directos y compartidos. Por el contrario, cada vez que se ejecuta un Fork, el sistema operativo debe crear un nuevo proceso con su propio espacio de direccionamiento independiente.

. Comparación de algoritmos de multiplicación de matrices

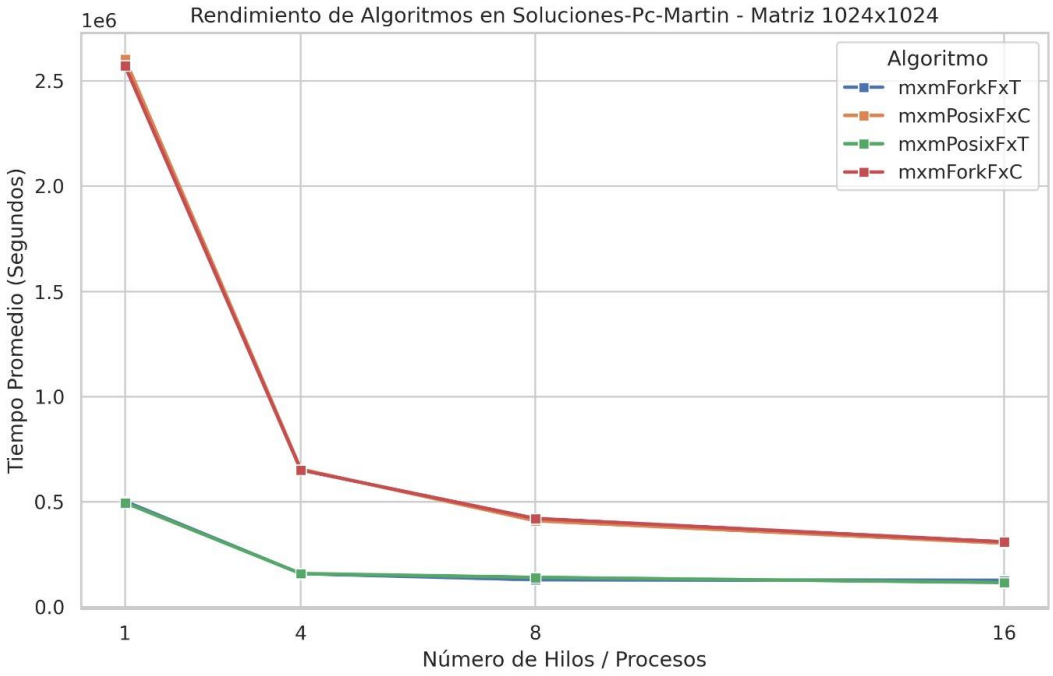

Figura 16:

Los cuatro algoritmos evaluados resultan de combinar dos decisiones independientes: el mecanismo de concurrencia (fork() vs pthreads) y la estrategia de partición del trabajo (por columnas FxC vs por filas FxT). Matemáticamente todos realizan el mismo número de operaciones, por lo que cualquier diferencia en tiempo de ejecución no viene del cómputo sino de cómo se accede a la memoria.

La multiplicación de matrices es un problema memory-bound: para calcular cada elemento de C se deben recorrer N elementos de una fila de A y N elementos de una columna de B, generando una presión enorme sobre la jerarquía de caché. Esto hace que el verdadero protagonista del experimento no sea el procesador sino la interacción entre el patrón de acceso del algoritmo y la jerarquía L1→L2→L3→RAM de cada equipo.

La diferencia más determinante del experimento es FxC vs FxT, y se explica completamente con los principios de localidad. En los algoritmos FxC, cada worker calcula columnas fijas de C, lo que obliga a acceder a B recorriendo sus columnas — en memoria row-major esto significa saltar N×4 bytes entre cada acceso.

Esto viola brutalmente la localidad espacial: cada línea de caché cargada (64 bytes = 16 floats) aporta solo 1 dato útil, desperdiciando el 93.75% del ancho de banda de memoria. Tampoco hay localidad temporal porque esos 15 floats "de regalo" en la línea cargada nunca se vuelven a usar antes de ser desalojados. El resultado es una avalancha de cache misses que con matrices de 2048×2048 se traduce en billones de ciclos esperando datos de RAM. En los algoritmos FxT, cada worker calcula filas completas de C, accediendo a A de forma perfectamente secuencial (máxima localidad espacial: todos los floats de cada línea de caché se usan) y reutilizando bloques de B repetidamente mientras permanecen en caché (localidad temporal activa). Esto le permite al hardware anticipar los accesos y ocultarlos, algo imposible con los saltos

irregulares de FxC. La consecuencia directa es visible en las gráficas: FxT es hasta 16 veces más rápido que FxC en los equipos con menos caché, y la brecha crece con N exactamente porque el problema de localidad se agrava cuanto más grande es la matriz.

La diferencia entre Fork y Pthreads, en cambio, resulta casi irrelevante en este experimento. fork() duplica el espacio de direcciones del proceso padre, mientras que pthread_create() crea un hilo que comparte el mismo espacio de direcciones con mucho menor overhead.

Sin embargo, con 1 CPU asignado no hay paralelismo real se turna el tiempo de búsqueda sobre el mismo núcleo y el tiempo total está completamente dominado por los fallos de caché, que representan el mayor porcentaje del tiempo de ejecución en los peores casos. El overhead de creación de procesos/hilos es un porcentaje minimo del tiempo total, imperceptible frente a todos los ciclos perdidos en accesos a RAM. La única diferencia sutil es que pthreads introduce riesgo de **false sharing** en FxC: dos hilos escribiendo columnas adyacentes de C pueden compartir líneas de caché y generar invalidaciones mutuas por el protocolo de coherencia MESI, lo que explica los tiempos marginalmente peores de PosixFxC respecto a ForkFxC en algunos equipos.

Conclusiones

Del análisis exhaustivo de los datos recolectados en el presente taller de evaluación de rendimiento, se extraen las siguientes conclusiones fundamentales:

1. **Localidad Espacial (FxT vs. FxC):** El algoritmo de Filas por Transpuestas (FxT) demostró una superioridad absoluta y consistente en todos los sistemas de cómputo evaluados. Esta ventaja radica en el respeto por la estructura Row-Major Order del lenguaje C, lo que maximiza la localidad espacial

2. **POSIX vs. Fork:** Si bien teóricamente la creación de procesos (Fork) acarrea un mayor overhead, se demostró empíricamente que puede ser competitivo en cargas medias gracias a optimizaciones del sistema operativo. Sin embargo, al escalar a matrices de 2048x2048, la necesidad de aislar y eventualmente duplicar masivamente la memoria provoca el colapso del sistema. Esto hace que POSIX como el mecanismo superior y definitivo para cargas de trabajo masivas que comparten estado.

3. **Penalización Overhead en Cargas Ligeras:** Se comprobó que la inyección de paralelismo no es una solución universal. En matrices de 512x512, el tiempo que el sistema operativo invierte en crear, gestionar y sincronizar las unidades de ejecución (overhead de concurrencia) supera ampliamente el beneficio computacional, resultando en ejecuciones paralelas menos eficientes que la ejecución puramente secuencial.

4. **Saturación Física y Ley de Amdahl:** Se verificó empíricamente el límite físico del hardware descrito por la Ley de Amdahl [8]. En los equipos con 4 u 8 núcleos lógicos, al solicitar 16 unidades de ejecución el rendimiento se estanca por completo (curva plana). El sistema operativo se ve forzado a gestionar las tareas mediante tiempo compartido (context switching), confirmando que solicitar más recursos por software de los que el hardware posee físicamente no genera ganancia alguna.

5. **El Impacto en hardware:** Las especificaciones de hardware determinaron el techo de escalabilidad. El PC-Martin (Intel Core i7-13650HX), apalancado por sus 20 núcleos y su memoria Caché L3 masiva (24 MiB), fue la única arquitectura capaz de mantener un paralelismo físico real con 16 unidades de ejecución, demostrando que para problemas de gran complejidad la cantidad de nucleos es fundamental

Recomendaciones

1. Usar siempre FxT sobre FxC para multiplicación de matrices en C, independientemente del sistema. La ganancia en rendimiento por el uso eficiente de caché justifica ampliamente el paso adicional de transponer la matriz.
2. Preferir POSIX Threads sobre Fork cuando se requiera concurrencia en operaciones con datos compartidos como lo es la multiplicación de matrices. Fork solo sería recomendable en escenarios donde el aislamiento total de memoria entre procesos sea un requisito de diseño.
3. Calibrar el número de hilos al número de núcleos lógicos disponibles. Superar ese límite no genera ganancia de rendimiento y añade overhead innecesario de gestión por parte del sistema operativo. Una buena práctica es consultar el número de núcleos con nproc antes de definir la concurrencia.
4. Evitar paralelismo en matrices pequeñas. Para dimensiones de 512×512 o menores, la ejecución secuencial es más eficiente. Se recomienda implementar un umbral de tamaño mínimo a partir del cual se active la concurrencia.
5. Considerar matrices de mayor dimensión (4096×4096) en máquinas con más RAM y núcleos como el PC-Martin, para observar si la escalabilidad con 16 hilos sigue mejorando o si aparecen nuevos cuellos de botella relacionados con la jerarquía de memoria.

REFERENCIAS

[1] Y. Oyama, S. Sasaki, S. Ishiguro, R. Matsumiya, J. Murakami, y O. Tatebe, "Reduction of Operating System Jitter Caused by Page Reclaim," en ROSS '14, Múnich, Alemania, 2014, ACM. Disponible en: http://dx.doi.org/10.1145/2612262.2612270 [Accedido: 29-abr-2026].

[2] Rafael Pérez Laserna, " La Ley de los Grandes Números," GeoGebra. [En línea]. Disponible en: https://www.geogebra.org/m/edy5fexu . [Accedido: 29-abr-2026].

[3]Weber State University, "Row-Major Order," *CS 1410 Textbook*. [En línea]. Disponible en: https://icarus.cs.weber.edu/~dab/cs1410/textbook/7.Arrays/row_major.html. [Accedido: 27-abr-2026].

[4] "Hilo en computación: ¿Qué es? | Lenovo México". Offizielle Website von Lenovo | Notebooks, Desktop-PCs, Tablets, Monitore | Lenovo DE. Accedido el 30 de abril de 2026. [En línea]. Disponible: https://www.lenovo.com/mx/es/glosario/hilo/

[5] Wikipedia, "Row- and column-major order," *Wikipedia, The Free Encyclopedia*. [En línea]. Disponible en: https://en.wikipedia.org/wiki/Row-_and_column-major_order. [Accedido: 27-abr-2026].

[6] W. Stallings, Sistemas operativos: Aspectos internos y principios de diseño, 7ma ed. Pearson Educación, 2012, cap. 3 - 4. [Accedido: 29-abr-2026].

[7] M. E. Rodríguez García, "Principios cuantitativos del diseño de computadoras," Universidad Tecnológica de la Mixteca, 2009. [En línea]. Disponible en: https://www.utm.mx/~merg/AC/2009/2.7-principios_cuantitativos.html. [Accedido: 28-abr-2026].

[8] EcuRed, "Ley de Amdahl," EcuRed. [En línea]. Disponible en: https://www.ecured.cu/Ley_de_Amdahl. [Accedido: 28-abr-2026].